



Native LEMP Tool

This script is a handful tool when you need a php enviroment (development or production) but with all the binaries installed natively in your operative system, no docker no vagrant, this is also a ideal tool to setup php servers on linux

lemptool is a rewrite of the the old dead [xnpm](#) bash script tool, with more reobust functionalities.

This script is compatible only for debian and ubuntu it uses [sury php repos](#) to allow you install multiple php versions on the same machine. The php installation option on this package also install commons php libraries for laravel development, and is intend for nginx with fpm usage.

Usage

Just clone the project and go to the project folder.

```
$ git clone https://github.com/kamansoft/native-lemp-tool.git
$ cd native-lemp-tool
```

Then jsut run the script as sudo with the -i argument and it will install all you need for a common php enviroment.

```
$ sudo ./lemptool -i
```

A set of things will happen then make sure to read every confirmation message before continuing.

You can also pass the --help argument in order to display some other options bundle in this script

```
$sudo ./lemptool --help
```

Usage:

```
lemptool [--option1 --option2 -y ...] [--param1=value1_2 value2_2 --param2=v
```

Option arguments are value less, instead with param arguments you can pass string values after the (=) sing.

Arguments:

-i, --install Proceed to setup repo and install:
nginx + php + mariadb + extra-laravel-developme
deb packages.

 -y You can pass this option to skip all the confirm
messages, is valid for all th below commands/op
-pki= --package-install= Installs os packages.

Usage example:

```
$ ./lemptool -pki=htop nodejs git
```

-ng --nginx-install Installs Nginx server.

-mdb --mariadb-install Installs Mariadb version 10.6.

 --mariadb-user-db-create Creates a new Mariadb user and a new db with th
name as user, also grant all rights to that use
the newly created database.

It needs a user and a password as parameters.

Usage example:

```
$ ./lemptool --mariadb-user-db-create=user pas
```

 --mariadb-superuser-create Creates a new Mariadb superuser. and gran all r
Usage example:

```
$ ./lemptool --mariadb-superuser-create=user p
```

-p= --php-install= Installs a php version on system together with
dependencies for laravel development,it also
installs composer.

When used the php version must be passed

as param value.

Usage example:

```
$ ./lemptool -p=8.0
```

`-fpm=` `--php-fpm-create=` Creates a new php fpm pool, using a template, it need a valid os user name as a param value.

Usage example:

```
$ ./lemptool -fpm=myusername
```

`--fpm-ng-server-create` Creates a new (laravel in mind), nginx server together with a existing valid fpm pool, a domain and local public files path to serve as params

Usage example:

```
$ ./lemptool --fpm-ng-server-create="fpm-name"
```

`-lemp` It takes a OS user create a folder at /var/www that user, it solflink it from user home folder runs `--fpm-ng-server-create` and `--mariadb-user`

Usage example:

```
$ ./lemptool lemp="username" "domain" "dbpasswo
```

`--phpmyadmin-install` Installs phpmyadmin from official source. It does so using fpm, so it create a php-fpm with a os 'phpmyadmin' user, also it create an nginx

Comprehensive Testing Framework

This project includes a robust three-tier testing framework that validates all functionalities across multiple operating systems:

Testing Levels

1. **Unit Tests** - Test individual functions in `lemptool_scripts`
2. **End-to-End Tests** - Test complete workflows and user scenarios
3. **Cross-OS Compatibility** - Validate across all supported operating systems



Supported Testing Platforms

- **Ubuntu:** 18.04, 20.04, 22.04, 24.04
- **Debian:** 11, 12



Quick Testing

```
# Run comprehensive tests across all OS versions
./testing/comprehensive-test.sh

# Run only unit tests across all operating systems
./testing/comprehensive-test.sh unit

# Run only end-to-end tests across all operating systems
./testing/comprehensive-test.sh e2e

# Test specific operating system
./testing/comprehensive-test.sh --os ubuntu22

# Run tests locally (without Docker)
./testing/comprehensive-test.sh local-unit
./testing/comprehensive-test.sh local-e2e
```



Test Coverage

- **Unit Tests:** 19 tests covering all core functions
- **E2E Tests:** 47 tests covering complete workflows
- **Cross-OS:** 100% compatibility across 6 operating systems
- **Success Rate:** 100% (123 assertions per OS)



Individual Test Scripts

```
# Direct unit testing
./testing/unit-tests.sh

# Direct end-to-end testing
./testing/e2e-tests.sh

# Docker-based testing for specific OS
./testing/docker-test.sh test ubuntu22 unit
./testing/docker-test.sh test debian12 e2e
./testing/docker-test.sh interactive ubuntu20
```

For comprehensive testing documentation, see [testing/README.md](#).

Development Guidelines



Testing Requirements

- **Always test changes** across multiple OS versions using the testing framework
- **Unit test coverage** for all new functions in `lemptool_scripts`
- **E2E test coverage** for all new workflows and user-facing features
- **Cross-OS validation** required before merging changes



Development Workflow

```
# 1. Run comprehensive tests before starting development
./testing/comprehensive-test.sh

# 2. Make your changes to the codebase
# ... edit files ...

# 3. Test specific functionality during development
./testing/unit-tests.sh           # Test function changes
./testing/e2e-tests.sh           # Test workflow changes

# 4. Test on specific OS if targeting specific compatibility
./testing/docker-test.sh interactive ubuntu22

# 5. Run full test suite before committing
./testing/comprehensive-test.sh all

# 6. Validate test results show 100% success rate
```



Code Quality Standards

- Follow bash best practices and coding guidelines
- Ensure container-safe operations (detect environment appropriately)
- Handle errors gracefully with meaningful messages
- Maintain backward compatibility across supported OS versions
- Document new functions and workflows



Debugging Guidelines

- Use interactive Docker sessions for debugging:
`./testing/docker-test.sh interactive [os]`
- Enable debug mode: `bash -x ./testing/comprehensive-test.sh`
- Check individual test outputs in `testing/` directory
- Validate cross-OS compatibility before finalizing changes

Compatibility Improvements

This project has been enhanced for full compatibility across all supported operating systems:

✓ Enhanced OS Support

- **Ubuntu:** 18.04 (Bionic), 20.04 (Focal), 22.04 (Jammy), 24.04 (Noble)
- **Debian:** 11 (Bullseye), 12 (Bookworm)

✓ Key Compatibility Features

- **Smart OS Detection** - Automatically detects OS version and adjusts behavior
- **Version-Aware Package Installation** - Uses compatible PHP/MariaDB versions per OS
- **Container-Safe Operations** - Works in Docker containers and bare metal
- **Enhanced Service Management** - Handles systemctl variations across environments
- **Safe User Creation** - Compatible user creation across all OS versions
- **Improved Error Handling** - Graceful fallbacks and better error messages

✓ Comprehensive Testing Framework

```
# Run complete test suite across all operating systems
./testing/comprehensive-test.sh all

# Test individual components
./testing/unit-tests.sh           # Function-level testing
./testing/e2e-tests.sh           # Workflow testing
./testing/docker-test.sh test ubuntu22  # OS-specific testing
```

Test Results: 100% success rate across all 6 operating systems

- **Unit Tests:** 19 tests, 35 assertions per OS
- **E2E Tests:** 47 tests, 88 assertions per OS
- **Total Coverage:** 66 tests, 123 assertions per OS version

✔ PHP Version Compatibility Matrix

OS Version	Compatible PHP Versions
Ubuntu 18.04	7.2, 7.3, 7.4, 8.0, 8.1, 8.2
Ubuntu 20.04	7.4, 8.0, 8.1, 8.2, 8.3
Ubuntu 22.04	8.0, 8.1, 8.2, 8.3
Ubuntu 24.04	8.0, 8.1, 8.2, 8.3
Debian 11	7.4, 8.0, 8.1, 8.2, 8.3
Debian 12	8.1, 8.2, 8.3

✔ MariaDB Version Compatibility

OS Version	Recommended MariaDB Version
Ubuntu 18.04	10.3
Ubuntu 20.04	10.5
Ubuntu 22.04	10.6
Ubuntu 24.04	11.0
Debian 11	10.5
Debian 12	10.11